PWM Breathing LED

1. Breathing Light Overview

PWM (Pulse Width Modulation) controls the average power by adjusting the time ratio of high and low levels. This experiment uses `machine.PWM` to drive the onboard LED (GPIO44). The duty cycle decreases from MAX (off) to VISIBLE (on) and then increases back to MAX, achieving a "breathing" effect of gradually brightening and dimming. The off phase additionally holds for 2s to form a complete breathing rhythm.

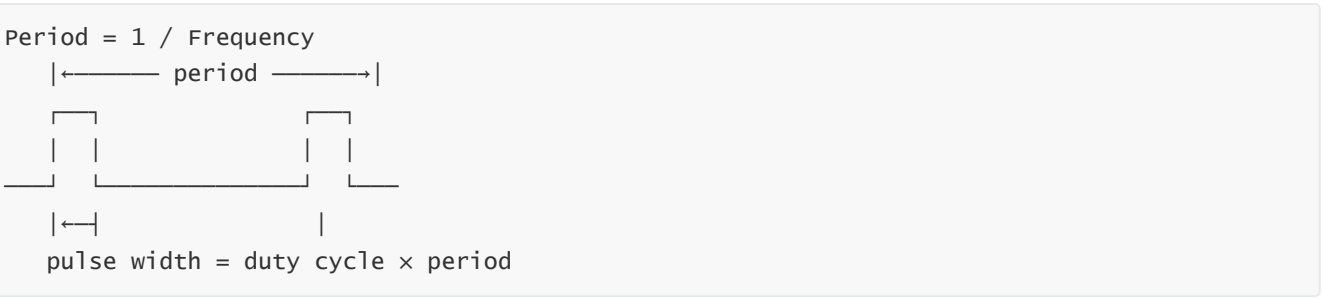
Typical application scenarios:

Application Type	Example
Brightness adjustment	LED dimming, backlight control
Motor control	Servo angle, DC motor speed control
Audio output	Buzzer tone control
Power management	DC-DC voltage regulation

2. Core Concepts

2.1 What is PWM

A PWM signal is defined by two parameters: **period** and **duty cycle**:



Average voltage = duty cycle × VDD.

2.2 PWM Parameter Selection

Parameter	Value in This Experiment	Description
Frequency (freq)	1000 Hz	Above the human flicker perception threshold (~100Hz)
Duty cycle range	0 ~ 1023	10-bit resolution (2^10 = 1024 levels)
MAX_DUTY	1023	Full high = off (common-anode)
VISIBLE	200	Brightness threshold; below this value the LED is visibly on

> If the frequency is too low (< 50Hz), flicker is perceptible to the human eye. 1kHz is the standard choice for LED dimming.

2.3 PWM Logic with a Common-Anode Connection

The onboard LED uses a **common-anode connection**: `duty=1023` (100% duty cycle, high output) = LED **off** (equal potential at both ends, no current), `duty=0` = LED **brightest**.

> With a common-anode connection, the PWM duty cycle is **inversely proportional** to brightness — duty↓ = on, duty↑ = off.

2.4 Core API

```
from machine import Pin, PWM

pwm = PWM(Pin(44))                  # create the PWM object
pwm.freq(1000)                       # set frequency to 1000Hz
pwm.duty(512)                        # set duty cycle 0~1023
pwm.deinit()                         # disable PWM
```

3. Hardware Dependencies

GPIO	Peripheral	Mode	Description
44	LED0	`PWM`	Onboard LED, common-anode, duty↑ = brightness↓

4. Project Architecture and Flow

```
Script execution (single-file .py)
  |
  ├─ Parameter config: MAX_DUTY=1023, VISIBLE=200, STEP=5, DELAY=12ms
  ├─ pwm = PWM(Pin(44), freq=1000, duty=1023)  // 1kHz, initially off
  |
  └─ while True:
       ├─ Off hold: pwm.duty(1023), sleep(2s)   // fully off, hold 2s
       |    └─ print("LED OFF (duty=1023)")
       ├─ Fade-in: for d in 1023→200 step -5:   // duty decreases (duty↓=brightness↑)
       |    ├─ print("Fade-in: duty 1023 → 200")
       |    ├─ pwm.duty(d)
       |    └─ sleep_ms(12)
       |    └─ print("LED ON (duty=200)")
       └─ Fade-out: for d in 200→1023 step 5:   // duty increases (duty↑=darkness↑)
            ├─ print("Fade-out: duty 200 → 1023")
            ├─ pwm.duty(d)
            └─ sleep_ms(12)
```

5. Source Code Details

Full source code: `5.Source Code → MicroPython → 1.Basic Peripherals → 7.PWM_LED → 7.PWM_LED.py`

5.1 Parameter Configuration and Initialization

```python
from machine import Pin, PWM  # Pin and PWM classes
import time                   # delay

# Configuration parameters
LED_PIN    = 44               # LED pin (common-anode)
PWM_FREQ   = 1000             # PWM frequency 1kHz
MAX_DUTY   = 1023             # 10-bit max duty cycle (=HIGH=OFF)
VISIBLE    = 200              # visible threshold (<this value=LOW=ON)
STEP       = 5                # duty step
DELAY_MS   = 12               # step delay
OFF_HOLD_S = 2                # OFF hold time (s)

# Init PWM: GPIO44, 1kHz, 10-bit
pwm = PWM(Pin(LED_PIN), freq=PWM_FREQ, duty=MAX_DUTY)
```

> `VISIBLE=200` is an empirical value — with a common-anode connection, the LED only becomes visibly on when the duty is below ~200. Above this value the LED still looks off, so fading in from MAX_DUTY down to VISIBLE is sufficient.

5.2 Breathing Loop

```
# Breathing LED
# Common-anode: LOW=ON (small duty), HIGH=OFF (large duty)

while True:
    # OFF phase: duty=MAX, hold for OFF_HOLD_S seconds
    pwm.duty(MAX_DUTY)
    print("LED OFF (duty=%d)" % MAX_DUTY)
    time.sleep(OFF_HOLD_S)

    # OFF→ON: duty decrement
    print("Fade-in: duty %d → %d" % (MAX_DUTY, VISIBLE))
    for d in range(MAX_DUTY, VISIBLE - 1, -STEP):
        pwm.duty(d)
        time.sleep_ms(DELAY_MS)
    print("LED ON (duty=%d)" % VISIBLE)

    # ON→OFF: duty increment
    print("Fade-out: duty %d → %d" % (VISIBLE, MAX_DUTY))
    for d in range(VISIBLE, MAX_DUTY + 1, STEP):
        pwm.duty(d)
        time.sleep_ms(DELAY_MS)
```

---

6. Operation Procedure

6.1 Flashing and Running

Thonny IDE steps: See the `1. Before Development → 3.Thonny IDE` chapter.

6.2 Serial Output Example

```
LED OFF (duty=1023)
Fade-in: duty 1023 → 200
LED ON (duty=200)
Fade-out: duty 200 → 1023
LED OFF (duty=1023)
Fade-in: duty 1023 → 200
...
```

```
LED OFF (duty=1023)
Fade-in: duty 1023 → 200
LED ON (duty=200)
Fade-out: duty 200 → 1023
LED OFF (duty=1023)
Fade-in: duty 1023 → 200
LED ON (duty=200)
Fade-out: duty 200 → 1023
```

6.3 Normal Phenomenon

The LED "breathes" slowly — off 2s → fade in → fade out → off 2s → repeat.

The TX blue indicator light blinks intermittently, indicating that the program is outputting print information through the serial port.

6.4 Verification Methods

Operation	**Expected Result**
Power up	Smooth breathing effect of the LED
Change `OFF_HOLD_S=2` to `OFF_HOLD_S=0`	No pause in the off phase, continuous breathing
Change `STEP=5` to `STEP=20`	Breathing noticeably faster
Change `DELAY_MS=12` to `DELAY_MS=50`	Breathing very slow, visible step changes

7. Common Problems

Symptom	**Cause**	**Solution**
LED does not light up	Wrong pin configuration	Confirm using `PWM(Pin(44))` instead of `Pin(44, Pin.OUT)`
Breathing not smooth, steppy	STEP too large or DELAY too short	Reduce STEP (e.g., 2) to increase the total number of steps
LED visibly flickers	Frequency too low	The frequency must be at least > 100Hz
`duty()` errors or has no effect	The pin is not initialized as PWM	Confirm `PWM(Pin(44))` instead of `Pin(44, Pin.OUT)`